

Plan de Implementacion — Soluciones Agenticas AMS

Documento vivo. El estatus de cada actividad se actualiza a medida que se completan. Fecha de inicio: 2026-04-06 | Branch: `feat/agentic-solutions`

Convenciones

Estatus	Significado
PENDIENTE	No iniciado
EN PROGRESO	En desarrollo
COMPLETADO	Terminado y verificado
BLOQUEADO	Dependencia no resuelta

Ejecucion	Significado
PARALELO	Se puede ejecutar simultaneamente con otras tareas del mismo bloque
SERIE	Requiere que las dependencias esten completadas antes de iniciar

Arquitectura General de una Solucion Agentica

Cada solucion sigue el mismo patron de 4 capas:

1. TOOL WRAPPER	→ <code>api/ai_core/tool_wrappers/{name}_tools.py</code> Funcion pura que invoca un engine o servicio Se registra via <code>@tool()</code> en el <code>TOOL_REGISTRY</code>
2. AGENT CONFIG	→ <code>api/ai_core/agents/{name}.py</code> (si es agente nuevo) O se reutiliza agente existente (<code>reliability</code> , <code>planning</code> , etc.) Se mapea en <code>AGENT_TOOL_MAP</code> en <code>server.py</code>
3. ROUTER/SERVICE	→ <code>api/routers/agentic.py</code> (nuevo router unificado) <code>api/services/agentic_{name}_service.py</code> Endpoint REST que orquesta la solucion
4. FRONTEND	→ <code>frontend/src/pages/Agentic{Name}.jsx</code> O widget embebido en pagina existente

FASE 0 — INFRAESTRUCTURA BASE

Crear el router unificado y los patrones reutilizables. Todo T1 depende de esto.

0.1 Router agentico unificado

Estatus	COMPLETADO
Ejecucion	PARALELO (con 0.2 y 0.3)
Archivo	<code>api/routers/agentic.py</code> (NUEVO)
Que hacer	Crear router FastAPI con prefijo <code>/api/v1/agentic/</code> . Registrar en <code>api/main.py</code> linea 26-42 importando <code>from api.routers import agentic</code> y anadiendo <code>app.include_router(agentic.router,</code> <code>prefix="/api/v1/agentic", tags=["agentic"])</code>
Patron	Cada solucion agentica sera un grupo de endpoints bajo este router

0.2 Modelo de AgenticExecution

Estatus	COMPLETADO
Ejecucion	PARALELO (con 0.1 y 0.3)
Archivo	api/database/models.py (agregar al final)
Que hacer	Crear <code>AgenticExecutionModel</code> para registrar cada ejecucion de solucion agentic: <code>execution_id</code> , <code>solution_type</code> (enum: VOICE_CAPTURE, AUTO_SCHEDULER, etc.), <code>triggered_by</code> (user_id), <code>plant_id</code> , <code>status</code> (RUNNING, COMPLETED, FAILED), <code>input_params</code> (JSON), <code>output_result</code> (JSON), <code>duration_ms</code> (int), <code>error_message</code> (str nullable), <code>created_at</code> , <code>completed_at</code> . Esto permite auditoria, metricas de uso, y debugging.

0.3 Servicio base agentic

Estatus	COMPLETADO
Ejecucion	PARALELO (con 0.1 y 0.2)
Archivo	api/services/agentic_base_service.py (NUEVO)
Que hacer	Crear funciones reutilizables: <code>start_execution(db, solution_type, user_id, plant_id, params) -> execution_id</code> , <code>complete_execution(db, execution_id, result)</code> , <code>fail_execution(db, execution_id, error)</code> . Wrapper que mide tiempo, persiste resultado, y maneja errores.

0.4 Registrar router en main.py

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 0.1)
Archivo	api/main.py linea 26-42
Que hacer	Anadir <code>agentic</code> al bloque de imports y registrar con <code>app.include_router()</code>

Verificacion Fase 0: Levantar servidor (`uvicorn api.main:app`) y confirmar que `/api/v1/agenti` responde con 200.

FASE 1 — TIER 1: VoiceCapture Pro (Solucion #1)

Depende de: Fase 0 completada

1.1 Endpoint de captura por voz unificado

Estatus	COMPLETADO
Ejecucion	PARALELO (con 1.2)
Archivo	api/routers/agenti.py
Que hacer	Crear endpoint <code>POST /agenti/voice-capture</code> . Request: <code>VoiceCaptureRequest(audio_base64: str None, image_base64: str None, text_input: str None, equipment_tag_hint: str None, plant_id: str, technician_id: str)</code> . Internamente llama a <code>agenti_voice_capture_service.process()</code> . Response: <code>{execution_id, work_request: dict, confidence: float, classification: dict}</code>
Reutilizar	<code>CaptureCreate</code> schema en <code>api/schemas.py</code> linea 73-99 como referencia para campos

1.2 Servicio de VoiceCapture

Estatus	COMPLETADO
Ejecucion	PARALELO (con 1.1)
Archivo	api/services/agentiv_voice_capture_service.py (NUEVO)
Que hacer	Orquestar el pipeline completo en una sola llamada:
Paso 1	Si audio_base64 : llamar transcripcion. Reutilizar logica de capture_service.py linea 161+ (process_capture). El texto transcrito se convierte en raw_text .
Paso 2	Si image_base64 : llamar vision_service.analyze() (ya existe en api/services/vision_service.py). Extraer: componente, anomalias, severidad.
Paso 3	Llamar /ai/suggest-failure internamente. Reutilizar ai_agents.py linea 610-666: la funcion construye un prompt con description + equipment_tag + equipment_name y llama Claude para clasificar: failure_category , symptom , cause , priority_suggested , work_order_type , estimated_duration_hours , required_specialties .
Paso 4	Crear WorkRequestModel con todos los campos pre-llenados. Reutilizar work_request_service.py : llamar compute_sla_deadline(priority) linea 14 y derive_work_class(priority) linea 21.
Paso 5	Crear FieldCaptureModel asociado al WR (source_capture_id).
Paso 6	Generar notificacion al supervisor. Reutilizar NotificationModel del models.py .
Return	{work_request_id, wo_number, priority, sla_deadline, classification, confidence}

1.3 Frontend — Widget de captura rapida

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 1.1 y 1.2)
Archivo	frontend/src/pages/mobile/MobileCrearWR.jsx (MODIFICAR)
Que hacer	Agregar boton "Captura Rapida por Voz" que: (a) graba audio via MediaRecorder API (ya usado en el campo <code>capture_type: "VOICE"</code>), (b) captura foto opcional via <code><input type="file" capture="camera"></code> , (c) envia a <code>POST /agentic/voice-capture</code> , (d) muestra resultado con opcion de editar antes de confirmar.
API client	Agregar en <code>frontend/src/api.js</code> : <code>voiceCapture: (data) => post('/agentic/voice-capture', data)</code>

1.4 Test E2E

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 1.3)
Archivo	tests/test_agentic_voice_capture.py (NUEVO)
Que hacer	Test con texto simulado (sin audio real): enviar <code>text_input="Bomba de pulpa vibrando excesivamente, ruido metalico en rodamiento lado libre"</code> , verificar que WR se crea con priority P2/P3, failure_category MECHANICAL, equipment_tag detectado.

Verificacion Solucion 1: Desde mobile, grabar audio describiendo un problema → ver WR creada automaticamente con clasificacion correcta.

FASE 2 — TIER 1: AutoScheduler (Solucion #2)

Depende de: Fase 0 completada Se puede ejecutar en **PARALELO** con Fase 1

2.1 Endpoint de auto-scheduling

Estatus	COMPLETADO
Ejecucion	PARALELO (con 2.2)
Archivo	api/routers/agentic.py
Que hacer	Crear endpoint <code>POST /agentic/auto-schedule</code> . Request: <code>AutoScheduleRequest(plant_id: str, week_number: int, year: int, include_preventive: bool = True, respect_shutdowns: bool = True)</code> . Llama a <code>agentic_scheduler_service.generate_program()</code> . Response: <code>{execution_id, program_id, program: dict, gantt: list, conflicts: list, material_status: dict, hh_balance: dict}</code>

2.2 Servicio de AutoScheduler

Estatus	COMPLETADO
Ejecucion	PARALELO (con 2.1)
Archivo	api/services/agentic_scheduler_service.py (NUEVO)
Que hacer	Orquestar en secuencia:
Paso 1	Llamar <code>scheduling_service.create_program(db, plant_id, week_number, year)</code> (linea 26-134 de <code>scheduling_service.py</code>). Esto ya hace: query backlog → filter schedulable → group via BacklogGrouper → create WeeklyProgram → level resources → detect conflicts.
Paso 2	Llamar <code>scheduling_service.check_materials(db, program_id)</code> (linea 343-348). Obtener estado de materiales.
Paso 3	Llamar <code>scheduling_service.hh_balance(db, program_id)</code> (linea 351-356). Obtener balance de horas-hombre.
Paso 4	Llamar <code>scheduling_service.get_gantt(db, program_id)</code> (linea 208-215). Obtener datos para Gantt.
Paso 5	Si hay conflictos criticos (over-allocation >120%), generar recomendaciones via Claude: "Dados estos conflictos de recursos: {conflicts}, sugiere redistribucion". Usar el patron de <code>ai_agents.py</code> linea 542-567 (<code>call_ai_tool</code>).
Return	Todo el paquete consolidado en un solo response.

2.3 Frontend — Boton "Generar Programa"

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 2.1 y 2.2)
Archivo	<code>frontend/src/pages/Scheduling.jsx</code> (MODIFICAR)
Que hacer	Agregar boton prominente "Generar Programa Automatico" en la pagina de Scheduling. Al hacer click: (a) muestra modal de confirmacion con semana/ano, (b) llama <code>POST /agentic/auto-schedule</code> , (c) muestra resultado con Gantt, conflictos, balance HH, (d) boton "Aprobar y Activar" que llama <code>PUT /scheduling/programs/{id}/activate</code> .
API client	Agregar en <code>api.js</code> : <code>autoSchedule: (data) => post('/agentic/auto-schedule', data)</code>

2.4 Export Excel del programa

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 2.3)
Archivo	Ya existe: <code>scheduling_service.export_gantt_excel()</code> linea 218-240
Que hacer	Exponer boton "Exportar a Excel" en el frontend que llama <code>GET /scheduling/programs/{id}/gantt/export</code> . Ya esta implementado como <code>FileResponse</code> . Solo agregar el boton en el UI.

Verificacion Solucion 2: Click en "Generar Programa" → programa semanal creado con Gantt, conflictos detectados, materiales verificados, exportable a Excel.

FASE 3 — TIER 1: EquipmentDoctor + SafetyChecklist (Soluciones #3 y #5)

Depende de: Fase 0 completada Se puede ejecutar en **PARALELO** con Fases 1 y 2

3.1 Endpoint EquipmentDoctor mejorado

Estatus	COMPLETADO
Ejecucion	PARALELO (con 3.2 y 3.3)
Archivo	api/routers/agentlic.py
Que hacer	Crear <code>POST /agentlic/equipment-doctor</code> . Request: <pre>EquipmentDoctorRequest(equipment_tag: str, symptom_description: str, plant_id: str, include_wr_suggestion: bool = True) .</pre> Internamente: (a) llamar <pre>equipment_chat_service.chat_with_equipment()</pre> (linea 194 de <code>equipment_chat_service.py</code>) con pregunta contextualizada de diagnostico, (b) si <code>confidence > 0.85</code> y <code>include_wr_suggestion=True</code> , pre-llenar WR draft via <code>work_request_service</code> . Response: <code>{diagnosis: str, confidence: float, suggested_wr: dict null, related_failure_modes: list, context_summary: dict}</code>

3.2 Servicio EquipmentDoctor

Estatus	COMPLETADO
Ejecucion	PARALELO (con 3.1 y 3.3)
Archivo	api/services/agentic_doctor_service.py (NUEVO)
Que hacer	Combinar:
Paso 1	Llamar equipment_chat_service._gather_equipment_con text(db, equipment_tag) (referenciar la funcion interna que recopila jerarquia, criticidad, FMEA, historial).
Paso 2	Construir prompt de diagnostico: "Eres un ingeniero de mantenimiento experto. Dado el siguiente contexto del equipo {context} y los sintomas reportados: {symptom_description}, proporciona: 1) Diagnostico probable con nivel de confianza, 2) Pasos de verificacion, 3) Si confianza > 85%, modo de falla probable y accion correctiva recomendada."
Paso 3	Llamar Claude API (modelo claude-sonnet-4-6, max_tokens: 500). Parsear respuesta para extraer confidence numericamente.
Paso 4	Si confidence > 0.85: crear WR draft con equipment_tag , priority derivada del diagnostico, failure_category del FMEA match. No persistir — retornar como sugerencia para que el tecnico confirme.

3.3 Endpoint SafetyChecklist mejorado

Estatus	COMPLETADO
Ejecucion	PARALELO (con 3.1 y 3.2)
Archivo	api/routers/agentic.py
Que hacer	Crear <code>POST /agentic/safety-checklist</code> . Request: <code>SafetyChecklistRequest(wo_id: str None, equipment_tag: str, task_type: str, plant_id: str)</code> . Reutiliza el endpoint existente <code>/ai/checklists</code> (linea 435-496 de <code>ai_agents.py</code>) pero agrega: (a) auto-deteccion de LOTO basado en <code>equipment_tag</code> y su jerarquia, (b) EPP especifico para altitud (4500m) y clima frio de Salares Norte, (c) historial de incidentes previos en el equipo. Response: <code>{checklist_id, sections: [{name, items: [{description, mandatory, completed}]}, safety_alerts: list}</code>

3.4 Frontend — Chat de diagnostico

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 3.1 y 3.2)
Archivo	frontend/src/pages/EquipmentChat.jsx (MODIFICAR)
Que hacer	Agregar modo "Diagnostico" al chat existente. Cuando el tecnico selecciona modo diagnostico: (a) campo de sintomas en lugar de pregunta libre, (b) llama <code>/agentic/equipment-doctor</code> , (c) muestra diagnostico con badge de confianza, (d) si hay WR sugerida, muestra card con boton "Crear Work Request".

3.5 Frontend — Checklist pre-ejecucion

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 3.3)
Archivo	frontend/src/pages/mobile/MobileTaskExecution.jsx (MODIFICADO — archivo real, plan referenciaba MobileTareaDetail.jsx que no existia)
Que hacer	Antes de permitir "Iniciar Tarea", mostrar boton "Generar Checklist de Seguridad" que llama /agentic/safety-checklist . Mostrar checklist con items marcables. No permitir inicio hasta que todos los items mandatory esten completados.

Verificacion Soluciones 3 y 5: Tecnico describe sintoma → diagnostico con confianza → WR sugerida. Antes de ejecutar OT → checklist generado con LOTO y EPP especifico.

FASE 4 — TIER 1: SmartBacklog (Solucion #4)

Depende de: Fase 0 completada Se puede ejecutar en **PARALELO** con Fases 1, 2, 3

4.1 Tool wrapper de scoring multi-criterio

Estatus	COMPLETADO
Ejecucion	PARALELO (con 4.2)
Archivo	api/ai_core/tool_wrappers/smart_backlog_tools.py (NUEVO)
Que hacer	Crear <code>@tool("score_backlog_item", ...)</code> que recibe un BacklogItem y calcula score compuesto: criticidad (25%, via <code>assess_criticality</code>), health_score (20%, via <code>calculate_health_score</code>), SLA proximity (20%, dias restantes / dias totales SLA), frecuencia de falla (15%, via <code>analyze_pareto</code>), costo de diferir (10%), impacto en seguridad (10%, del FMEA consequence). Retorna <code>{total_score: float, breakdown: dict, alerts: list}</code> .
Registrar	En <code>server.py</code> : <code>import</code> <code>api.ai_core.tool_wrappers.smart_backlog_tools</code> y agregar <code>"score_backlog_item"</code> al <code>AGENT_TOOL_MAP["planning"]</code> .

4.2 Servicio SmartBacklog

Estatus	COMPLETADO
Ejecucion	PARALELO (con 4.1)
Archivo	api/services/agentik_smart_backlog_service.py (NUEVO)
Que hacer	Funcion <code>prioritize_backlog(db, plant_id) -> dict</code> :
Paso 1	Cargar todos los <code>BacklogItemModel</code> con status READY.
Paso 2	Para cada item, calcular score multi-criterio (llamar tool wrapper o calcular inline).
Paso 3	Ordenar por score descendente.
Paso 4	Identificar items con SLA_breach_risk (dias restantes < 20% del SLA total).
Paso 5	Agrupar por equipo/area usando <code>BacklogGrouper.find_all_groups()</code> (ya existe en <code>backlog_tools.py</code>).
Paso 6	Retornar: <code>{ranked_items: list, sla_alerts: list, grouping_suggestions: list, stats: {total, critical, high, normal, low, avg_age_days}}</code>

4.3 Endpoint SmartBacklog

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 4.1 y 4.2)
Archivo	api/routers/agentik.py
Que hacer	POST <code>/agentik/smart-backlog</code> con <code>SmartBacklogRequest(plant_id: str)</code> . Retorna el resultado del servicio.

4.4 Frontend — Vista de backlog inteligente

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 4.3)
Archivo	frontend/src/pages/Backlog.jsx (MODIFICAR)
Que hacer	Agregar toggle "Vista Inteligente" que reemplaza la tabla por la vista priorizada del SmartBacklog. Mostrar: score como barra de progreso coloreada, alertas SLA como badges rojos, sugerencias de agrupacion como cards colapsables.

Verificacion Solucion 4: Activar "Vista Inteligente" → backlog reordenado por score, alertas SLA visibles, sugerencias de agrupacion.

FASE 5 — TIER 1: KPI Watchdog (Solucion #6)

Depende de: Fase 0 completada Se puede ejecutar en **PARALELO** con Fases 1-4

5.1 Servicio KPI Watchdog

Estatus	COMPLETADO
Ejecucion	PARALELO (con 5.2)
Archivo	api/services/agentik_kpi_watchdog_service.py (NUEVO)
Que hacer	Funcion <code>run_watchdog(db, plant_id) -> dict :</code>
Paso 1	Cargar ultimos 90 dias de <code>KPIMetricsModel</code> snapshots para la planta.
Paso 2	Calcular media movil y desviacion estandar para cada KPI (MTBF, MTTR, OEE, availability, schedule_compliance, pm_compliance, reactive_ratio).
Paso 3	Para KPIs actuales que desvian $>2\sigma$, llamar <code>detect_variance</code> tool (ya existe en <code>variance_tools.py</code>).
Paso 4	Para cada desviacion detectada, correlacionar con OTs recientes del area/equipo afectado (query <code>ManagedWorkOrderModel</code> filtrado por <code>equipment_tag</code> y fecha).
Paso 5	Generar analisis causal via Claude: "KPI {name} ha caido de {baseline} a {current} en area {area}. OTs recientes relacionadas: {wo_list}. Cual es la causa probable y que accion correctiva recomiendas?"
Paso 6	Crear <code>NotificationModel</code> para cada alerta (ya existe modelo).
Return	<code>{alerts: [{kpi, current_value, baseline, deviation_sigma, area, probable_cause, recommended_action, severity}], summary: str}</code>

5.2 Endpoint KPI Watchdog

Estatus	COMPLETADO
Ejecucion	PARALELO (con 5.1)
Archivo	api/routers/agentic.py
Que hacer	POST /agentic/kpi-watchdog con KPIWatchdogRequest(plant_id: str) . Retorna alertas y resumen. Tambien: GET /agentic/kpi-watchdog/history para ver alertas historicas.

5.3 Frontend — Panel de alertas KPI

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 5.1 y 5.2)
Archivo	frontend/src/components/views/ExecutiveView.jsx (MODIFICAR)
Que hacer	Agregar seccion "Alertas KPI" en el dashboard ejecutivo. Mostrar cards con semaforo (rojo/amarillo/verde) por KPI, tendencia (sparkline), y accion recomendada expandible. Boton "Ejecutar Watchdog" que llama al endpoint.

Verificacion Solucion 6: Ejecutar watchdog → alertas generadas para KPIs fuera de rango con causa probable y accion recomendada.

FASE 6 — TIER 1: Executive Report (Solucion #7)

Depende de: Fase 0 completada Se puede ejecutar en **PARALELO** con Fases 1-5

6.1 Servicio de Reporte Ejecutivo

Estatus	COMPLETADO
Ejecucion	PARALELO (con 6.2)
Archivo	api/services/agentice_executive_report_service.py (NUEVO)
Que hacer	Funcion generate_executive_report(db, plant_id, period="monthly") -> dict :
Paso 1	Llamar reporting_service.generate_weekly_report() o generate_monthly_kpi_report() (ya existen como tools: generate_weekly_report , generate_monthly_kpi_report en reporting_tools.py).
Paso 2	Llamar KPI engine para metricas actuales (MTBF, MTTR, OEE, disponibilidad).
Paso 3	Llamar backlog service para estado del backlog (total, por prioridad, aging).
Paso 4	Llamar budget data: query ManagedWorkOrderModel para sumar actual_total_cost vs budget_amount por area.
Paso 5	Llamar health_score para top 10 equipos criticos.
Paso 6	Generar resumen ejecutivo via Claude: "Genera un resumen ejecutivo de 3 parrafos sobre el desempeno de mantenimiento del periodo. KPIs: {kpis}. Backlog: {backlog}. Presupuesto: {budget}. Equipos criticos: {health}."
Paso 7	Estructurar datos para PowerPoint: slides list con titulo, tipo (kpi_card, chart, table, text), y datos.
Return	{summary_text, slides_data, kpis, backlog_stats, budget_variance, critical_equipment}

6.2 Endpoint de generacion PPTX

Estatus	COMPLETADO
Ejecucion	PARALELO (con 6.1)
Archivo	api/routers/agentic.py
Que hacer	POST /agentic/executive-report retorna JSON con datos estructurados. POST /agentic/executive-report/pptx retorna FileResponse con archivo .pptx generado via python-pptx (ya en requirements.txt). POST /agentic/executive-report/xlsx retorna Excel via openpyxl (ya en requirements.txt).

6.3 Generador PPTX

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 6.1)
Archivo	api/services/pptx_generator_service.py (NUEVO)
Que hacer	Funcion generate_pptx(report_data: dict) -> str (filepath) : Crear presentacion con python-pptx . Slide 1: Titulo + periodo. Slide 2: KPIs con semaforos (tabla 2x4). Slide 3: Tendencias (insertar charts como imagenes via plotly → png → pptx). Slide 4: Backlog (tabla). Slide 5: Presupuesto (barras). Slide 6: Top 10 equipos criticos. Slide 7: Resumen ejecutivo (texto generado por Claude). Slide 8: Proximas acciones.

6.4 Frontend — Boton de reporte

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 6.2 y 6.3)
Archivo	frontend/src/components/views/ExecutiveView. jsx (MODIFICAR)
Que hacer	Agregar boton "Generar Reporte Ejecutivo" con dropdown: "Ver en pantalla", "Descargar PPTX", "Descargar Excel".

Verificacion Solucion 7: Click "Generar Reporte" → vista previa en pantalla + descarga PPTX con 8 slides profesionales.

FASE 7 — TIER 2: Chronic Failure Detector (Solucion #8)

Depende de: Fase 0 completada Se puede ejecutar en **PARALELO** con Fases 1-6

7.1 Servicio Detector de Fallas Cronicas

Estatus	COMPLETADO
Ejecucion	PARALELO (con 7.2)
Archivo	api/services/agentic_chronic_failure_service.py (NUEVO)
Que hacer	Funcion detect_chronic_failures(db, plant_id, lookback_months=12) -> dict :
Paso 1	Query todas las OTs cerradas (ManagedWorkOrderModel con status CERRADO) de los ultimos lookback_months .
Paso 2	Agrupar por equipment_tag + wo_type . Identificar equipos con >= 3 OTs correctivas en el periodo.
Paso 3	Para cada equipo cronico: llamar tool analyze_pareto (ya existe en planning tools) para ranking.
Paso 4	Para top 10 cronicos: llamar tool fit_weibull (ya existe en weibull_tools.py) con intervalos entre fallas.
Paso 5	Calcular costo acumulado por equipo (sum de actual_total_cost).
Paso 6	Generar resumen y recomendacion via Claude para cada equipo cronico.
Return	{chronic_items: [{equipment_tag, failure_count, avg_interval_days, weibull_beta, weibull_eta, annual_cost, trend, recommendation}], total_chronic_cost, top_5_summary}

7.2 Endpoint y frontend

Estatus	COMPLETADO
Ejecucion	PARALELO (con 7.1)
Archivo	api/routers/agentic.py + frontend/src/pages/Reliability.jsx
Que hacer	Endpoint: POST /agentic/chronic-failures . Frontend: agregar tab "Fallas Cronicas" en pagina de Reliability con tabla de equipos cronicos, graficos Pareto, y parametros Weibull.

Verificacion Solucion 8: Ejecutar detector → lista de equipos cronicos con Pareto, Weibull, costo anual, y recomendaciones.

FASE 8 — TIER 2: Material Readiness + WO Router (Soluciones #9 y #15)

Depende de: Fase 0 completada

8.1 Servicio Material Readiness

Estatus	COMPLETADO
Ejecucion	PARALELO (con 8.2)
Archivo	api/services/agentic_material_service.py (NUEVO)
Que hacer	Reutilizar la logica de scheduling_service.check_materials() (linea 343-485) pero aplicada a nivel de item individual. Para cada item del backlog: verificar materials JSON array del WR asociado contra inventario (InventoryItemModel). Retornar: {item_id, materials_status: "ready" "partial" "missing", missing_items: [{code, description, qty_needed, qty_available, lead_time_days}]}

8.2 Servicio WO Router Inteligente

Estatus	COMPLETADO
Ejecucion	PARALELO (con 8.1)
Archivo	api/services/agentic_wo_router_service.py (NUEVO)
Que hacer	Funcion <code>route_work_request(db, work_request_id) -> dict :</code>
Paso 1	Cargar WR y su <code>ai_classification</code> .
Paso 2	Buscar OTs historicas similares: query <code>ManagedWorkOrderModel</code> where <code>equipment_tag</code> = WR.equipment_tag AND status = CERRADO, ultimas 20. Calcular promedio <code>actual_hours</code> .
Paso 3	Determinar specialties requeridas de la clasificacion AI o del historico.
Paso 4	Llamar <code>assignment_service.optimize_assignments()</code> (linea 75-97 de <code>assignment_service.py</code>) para sugerir crew optima.
Paso 5	Verificar materiales via servicio 8.1.
Paso 6	Pre-llenar OT draft: <code>managed_wo_service.create_work_order()</code> con todos los campos pero status CREADO (no fast-track).
Return	<code>{draft_wo: dict, suggested_crew: list, estimated_hours: float, materials_status: dict, similar_wos: list}</code>

8.3 Endpoints y frontend

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 8.1 y 8.2)
Archivo	api/routers/agentic.py + frontend
Que hacer	Endpoints: POST /agentic/material-readiness/{backlog_id} , POST /agentic/router/{request_id} . Frontend: en pagina de Work Requests, agregar boton "Planificar con IA" que llama al WO Router y muestra el draft pre-llenado para revision.

Verificacion Soluciones 9 y 15: Aprobar WR → click "Planificar con IA" → OT draft pre-llenada con crew, materiales, y duracion estimada.

FASE 9 — TIER 2: Shift Handover + RCM Advisor (Soluciones #11 y #10)

Depende de: Fase 0 completada

9.1 Servicio Shift Handover

Estatus	COMPLETADO
Ejecucion	PARALELO (con 9.2)
Archivo	api/services/agentic_handover_service.py (NUEVO)
Que hacer	Funcion generate_handover(db, plant_id, shift_date, shift_type) -> dict :
Paso 1	Query OTs con actual_start en el turno (ej: MORNING = 07:00-19:00). Separar en: completadas, en progreso, iniciadas.
Paso 2	Query WorkAssignmentModel para progreso por tecnico.
Paso 3	Query NotificationModel de tipo seguridad del turno.
Paso 4	Query backlog items con SLA_breach inminente (< 24h).
Paso 5	Generar resumen estructurado via Claude.
Return	{shift_summary, completed_wos, in_progress_wos, attention_flags, safety_incidents, pending_materials, notes_for_next_shift}

9.2 Servicio RCM Advisor

Estatus	COMPLETADO
Ejecucion	PARALELO (con 9.1)
Archivo	api/services/agentric_rcm_advisor_service.py (NUEVO)
Que hacer	Funcion <code>advise_rcm_strategy(db, equipment_tag, plant_id) -> dict :</code>
Paso 1	Cargar failure modes del equipo (query <code>FailureModeModel</code> filtrado por equipment hierarchy node).
Paso 2	Para cada failure mode: llamar tool <code>rcm_decide</code> (ya existe en <code>rcm_tools.py</code>).
Paso 3	Generar lista de tareas recomendadas con intervalos via tool <code>generate_work_instruction</code> .
Paso 4	Calcular confianza basada en completitud de FMEA data.
Return	<code>{equipment_tag, failure_modes_analyzed: int, strategies: [{fm_code, mechanism, cause, rcm_decision, strategy_type, recommended_task, interval, confidence}], draft_task_list: list}</code>

9.3 Endpoints y frontend

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 9.1 y 9.2)
Archivo	api/routers/agentric.py + frontend
Que hacer	Endpoints: <code>POST /agentric/shift-handover</code> , <code>POST /agentric/rcm-advisor</code> . Frontend: Handover como pagina accesible al Supervisor con generacion automatica al fin de turno. RCM Advisor como tab en pagina FMEA.

FASE 10 — TIER 2: Budget Sentinel + SAP Sync + Defect Elimination (Soluciones #13, #12, #16)

Depende de: Fase 0 completada

10.1 Budget Sentinel

Estatus	COMPLETADO
Ejecucion	PARALELO (con 10.2 y 10.3)
Archivo	api/services/agentic_budget_sentinel_service.py (NUEVO, 377 lineas)
Que hacer	Query ManagedWorkOrderModel para sumar costos por area/mes. Comparar con presupuesto (query annual_budget o seed_data). Proyectar cierre de mes con run rate. Generar alertas amarillo/naranja/rojo segun umbrales (90%/100%/110%).

10.2 SAP Sync Guardian

Estatus	COMPLETADO
Ejecucion	PARALELO (con 10.1 y 10.3)
Archivo	api/services/agentic_sap_sync_service.py (NUEVO, 235 lineas)
Que hacer	Reutilizar sap_rfc_connector.py (1400 lineas). Funcion check_sync_health(db) : verificar cola de SAPUploadPackageModel , validar paquetes pendientes via validate_sap_field_lengths y validate_sap_cross_references (ya existen como tools). Generar reporte de discrepancias.

10.3 Defect Elimination Tracker

Estatus	COMPLETADO
Ejecucion	PARALELO (con 10.1 y 10.2)
Archivo	api/services/agentict_defect_elimination_service.py (NUEVO, 282 lineas)
Que hacer	Reutilizar tools CAPA existentes: create_capa , advance_capa_phase , check_capa_overdue (en capa_tools.py). Funcion track_defect_elimination(db, plant_id) : cargar RCAs completados, verificar que cada uno tiene CAPA asociado, identificar CAPAs vencidos, calcular DE KPIs via tool calculate_de_kpis_standalone .

10.4 Post-Maintenance Learning (Solucion #14)

Estatus	COMPLETADO
Ejecucion	PARALELO (con 10.1-10.3)
Archivo	api/services/agentict_learning_service.py (NUEVO, 396 lineas)
Que hacer	Al cerrar OT: comparar estimated_hours vs actual_hours , analizar si failure mode fue correcto, actualizar estadisticas de precision. Almacenar como PostMaintenanceLearning en un JSON field del WO o en modelo nuevo.

10.5 Endpoints y frontend para Fase 10

Estatus	COMPLETADO
Ejecucion	SERIE (depende de 10.1-10.4)
Archivo	api/routers/agentict.py + frontend
Que hacer	Endpoints: POST /agentict/budget-sentinel , POST /agentict/sap-sync-check , POST /agentict/defect-tracker , POST /agentict/post-learning/{wo_id} . Frontend: widgets en dashboards respectivos.

FASE 11 — TIER 3: Predictive Health + Shutdown Optimizer

(Soluciones #17 y #18)

Depende de: Fases 5 y 7 completadas (necesita KPI historico y Weibull)

11.1 Predictive Health Prophet

Estatus	COMPLETADO
Ejecucion	PARALELO (con 11.2)
Archivo	api/services/agentic_predictive_health_service.py (NUEVO)
Que hacer	Para cada equipo critico (risk_class A/B): calcular health_score actual, ajustar Weibull con historial de fallas, predecir ventana de falla probable a 30/60/90 dias. Rankear por probabilidad de falla. Generar plan proactivo de intervenciones. Reutilizar: fit_weibull , predict_failure , weibull_reliability , calculate_health_score (todos ya existen como tools).

11.2 Shutdown Optimizer

Estatus	COMPLETADO
Ejecucion	PARALELO (con 11.1)
Archivo	api/services/agentic_shutdown_optimizer_service.py (NUEVO)
Que hacer	Reutilizar tools existentes: create_shutdown , calculate_shutdown_metrics . Agregar: agrupacion de backlog items con shutdown_required=True , secuenciacion por ruta critica (topological sort basado en dependencias de equipo), nivelacion de recursos multi-especialidad, validacion de materiales.

11.3 Endpoints y frontend

Estatus	PENDIENTE
Ejecucion	SERIE (depende de 11.1 y 11.2)

FASE 12 — TIER 3: Remaining Solutions (Soluciones #19-26)

Depende de: Fases previas

12.1 Compliance Watchdog (#19)

Estatus	COMPLETADO
Ejecucion	PARALELO
Archivo	api/services/agentic_compliance_service.py (NUEVO)
Que hacer	Reutilizar tools: <code>check_rbi_overdue</code> , <code>check_kpi_breaches</code> . Agregar calendario regulatorio SERNAGEOMIN.

12.2 Knowledge Base Curator (#21)

Estatus	COMPLETADO
Ejecucion	PARALELO
Archivo	api/services/agentic_knowledge_service.py (NUEVO)
Que hacer	Post-cierre de troubleshooting sessions y RCAs, extraer conocimiento y almacenar en <code>ExpertCardModel</code> (ya existe). Reutilizar <code>expert_knowledge</code> router.

12.3 Spare Parts Forecaster (#22)

Estatus	PENDIENTE
Ejecucion	PARALELO
Archivo	api/services/agentic_spare_parts_forecast_service.py (NUEVO)
Que hacer	Reutilizar tools: analyze_spare_parts , calculate_stock_levels . Agregar proyeccion a 3/6/12 meses basada en consumo historico y Weibull.

12.4 Autonomous RCA Initiator (#26)

Estatus	PENDIENTE
Ejecucion	PARALELO
Archivo	api/services/agentic_rca_initiator_service.py (NUEVO)
Que hacer	Trigger: cuando OT P1/P2 se cierra, automaticamente llamar classify_rca_event , create_rca_analysis , run_rca_5w2h (todos existen como tools en rca_tools.py). Pre-llenar con historial de fallas similares.

12.5 Contractor Performance (#23) + Multi-Site Benchmarking (#25)

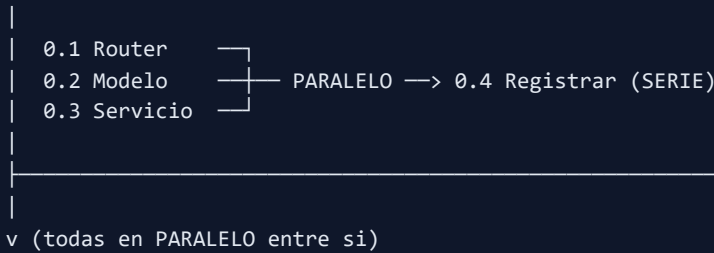
Estatus	PENDIENTE
Ejecucion	PARALELO
Que hacer	Agregar datos de OTs por contratista. Usar detect_variance y rank_plants tools para benchmarking.

12.6 Digital Twin (#20) + Energy Monitor (#24)

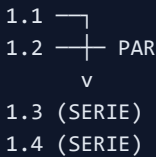
Estatus	PENDIENTE
Ejecucion	PARALELO
Que hacer	Frontend pesado: mapa visual de planta con health scores en tiempo real. Requiere integracion PI historian para energia.

Diagrama de Dependencias y Paralelismo

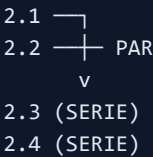
FASE 0: Infraestructura Base



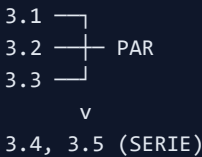
FASE 1: VoiceCapture



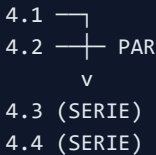
FASE 2: AutoScheduler



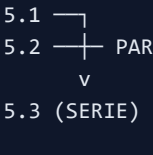
FASE 3: Doctor+Checklist



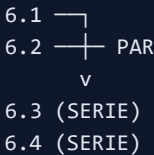
FASE 4: SmartBacklog



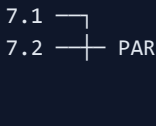
FASE 5: KPI Watchdog



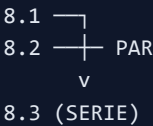
FASE 6: Exec Report



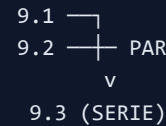
FASE 7: Chronic Failure



FASE 8: Materials+Router



FASE 9: Handover+RCM

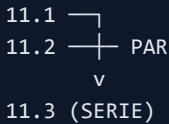


FASE 10: Budget+SAP+CAPA+Learning (todas sub-tareas en PARALELO)

10.1, 10.2, 10.3, 10.4 — PAR —> 10.5 (SERIE)



FASE 11: Predictive+Shutdown



FASE 12: Remaining T3

12.1-12.6 — PARALELO

Resumen de Archivos a Crear/Modificar

Archivos NUEVOS (crear)

#	Archivo	Fase	Tipo
1	api/routers/agentic.py	0	Router unificado
2	api/services/agentic_base_service.py	0	Servicio base
3	api/services/agentic_voice_capture_service.py	1	VoiceCapture
4	api/services/agentic_scheduler_service.py	2	AutoScheduler
5	api/services/agentic_doctor_service.py	3	EquipmentDoctor
6	api/services/agentic_smart_backlog_service.py	4	SmartBacklog
7	api/ai_core/tool_wrappers/smart_backlog_tools.py	4	Tool wrapper
8	api/services/agentic_kpi_watchdog_service.py	5	KPI Watchdog
9	api/services/agentic_executive_report_service.py	6	Executive Report
10	api/services/pptx_generator_service.py	6	Generador PPTX
11	api/services/agentic_chronic_failure_service.py	7	Chronic Failure
12	api/services/agentic_material_service.py	8	Material Readiness

#	Archivo	Fase	Tipo
13	api/services/agentic_wo_router_service.py	8	WO Router
14	api/services/agentic_handover_service.py	9	Shift Handover
15	api/services/agentic_rcm_advisor_service.py	9	RCM Advisor
16	api/services/agentic_budget_sentinel_service.py	10	Budget Sentinel
17	api/services/agentic_sap_sync_service.py	10	SAP Sync
18	api/services/agentic_defect_elimination_service.py	10	Defect Elimination
19	api/services/agentic_learning_service.py	10	Post-Maint Learning
20	api/services/agentic_predictive_health_service.py	11	Predictive Health
21	api/services/agentic_shutdown_optimizer_service.py	11	Shutdown Optimizer
22-27	api/services/agentic_{remaining}_service.py	12	T3 solutions
28	tests/test_agentic_voice_capture.py	1	Test
29	tests/test_agentic_scheduler.py	2	Test
30	tests/test_agentic_distributor.py	3	Test

Archivos EXISTENTES a modificar

#	Archivo	Fase	Cambio
1	api/main.py	0	Agregar import y include_router para agentic
2	api/database/models.py	0	Agregar AgenticExecutionMode 1
3	api/ai_core/tool_wrappers/server.py	4	Import smart_backlog_tools, agregar a AGENT_TOOL_MAP
4	frontend/src/api.js	1-6	Agregar funciones API para cada solucion
5	frontend/src/pages/mobile/MobileCrearWR.jsx	1	Widget captura rapida
6	frontend/src/pages/Scheduling.jsx	2	Boton auto-schedule
7	frontend/src/pages/EquipmentChat.jsx	3	Modo diagnostico
8	frontend/src/pages/mobile/MobileTareaDetail.jsx	3	Checklist pre-ejecucion
9	frontend/src/pages/Backlog.jsx	4	Vista inteligente
10	frontend/src/components/views/ExecutiveView.jsx	5,6	Alertas KPI + reporte
11	frontend/src/pages/Reliability.jsx	7	Tab fallas cronicas

Orden de Ejecucion Recomendado

Semana 1: FASE 0 (infra) + iniciar FASE 1, 2, 3 en paralelo
Semana 2: Completar FASE 1, 2, 3 + iniciar FASE 4, 5, 6 en paralelo
Semana 3: Completar FASE 4, 5, 6 + tests de integracion T1
Semana 4: FASE 7, 8 en paralelo (T2 comienza)
Semana 5: FASE 9, 10 en paralelo
Semana 6: Completar T2 + tests de integracion T2
Semana 7-8: FASE 11, 12 (T3)

Mapa de Ejecucion Optima — Vista Consolidada

Lectura: cada fila es un slot de tiempo. Las columnas muestran que se ejecuta simultaneamente. Las flechas >>> indican dependencia en serie (la tarea de la derecha espera a la izquierda). Las tareas en la misma fila sin flecha son **paralelo puro**.

FASE 0 — Infraestructura (Dia 1)

SLOT	CANAL A	CANAL B	CANAL C
1	0.1 Router agentic.py [NUEVO] PARALELO	0.2 AgenticExecutionModel [models.py] PARALELO	0.3 Servicio base [NUEVO] PARALELO
2	0.4 Registrar en main.py SERIE >>> espera 0.1		
	VERIFICAR: GET /api/v1/agentic/ → 200 OK		

TIER 1 — Fases 1-6 en paralelo (Dias 2-10)

Las 6 fases del Tier 1 se pueden ejecutar en **6 canales paralelos**. Dentro de cada fase, backend y frontend van en serie (frontend espera al backend).

SLOT	FASE 1	FASE 2	FASE 3	FASE 4
FASE 5	FASE 6			
VoiceCapture	AutoScheduler	Doctor+Checklist	SmartBacklog	
KPI Watchdog	Exec Report			
+-----+-----+-----+-----+-----				
3	1.1 Endpoint	2.1 Endpoint	3.1 Endpoint Doctor	4.1 Tool wrapper
5.1 Servicio	6.1 Servicio			
1.2 Servicio	2.2 Servicio	3.2 Servicio Doctor	4.2 Servicio	
5.2 Endpoint	6.2 Endpoint			
PARALELO 1.1 y 1.2	PARALELO 2.1 y 2.2	3.3 Endpoint Checklist	PARALELO 4.1 y 4.2	
PARALELO 5.1 y 5.2	PARALELO 6.1 y 6.2			
		PARALELO 3.1-3.3		
+-----+-----+-----+-----+-----				
4	1.3 Frontend mobile	2.3 Frontend btn	3.4 Frontend chat	4.3 Endpoint
5.3 Frontend panel	6.3 PPTX generator			
SERIE >>> 1.1+1.2	SERIE >>> 2.1+2.2	3.5 Frontend checklist	SERIE >>> 4.1+4.2	
SERIE >>> 5.1+5.2	SERIE >>> 6.1			
		SERIE >>> 3.1-3.3		
+-----+-----+-----+-----+-----				
5	1.4 Test E2E	2.4 Export Excel		4.4 Frontend vista
	6.4 Frontend btn			
SERIE >>> 1.3	SERIE >>> 2.3		SERIE >>> 4.3	
	SERIE >>> 6.3			
+-----+-----+-----+-----+-----				
+-----+-----+-----+-----+-----				
	VERIFICAR T1 COMPLETO: 7 soluciones funcionando end-to-end			

TIER 2 — Fases 7-10 en paralelo (Dias 11-18)

Las 4 fases del Tier 2 se ejecutan en **4 canales paralelos**.

SLOT	FASE 7	FASE 8	FASE 9	
FASE 10	Chronic Failure	Materials + W0 Router	Handover + RCM Advisor	
Budget+SAP+CAPA+Learning				
6	7.1 Servicio	8.1 Svc Material	9.1 Svc Handover	
10.1 Budget Sentinel	7.2 Endpoint + Frontend	8.2 Svc W0 Router	9.2 Svc RCM Advisor	
10.2 SAP Sync	PARALELO 7.1 y 7.2	PARALELO 8.1 y 8.2	PARALELO 9.1 y 9.2	10.3
Defect Elimination				
10.4 Post-Maint Learning				
PARALELO 10.1-10.4				
7		8.3 Endpoints + Frontend	9.3 Endpoints + Frontend	
10.5 Endpoints + Frontend		SERIE >>> 8.1+8.2	SERIE >>> 9.1+9.2	SERIE
>>> 10.1-10.4				
	VERIFICAR T2 COMPLETO: 9 soluciones adicionales funcionando			

TIER 3 — Fases 11-12 en paralelo (Dias 19-28)

SLOT	FASE 11	FASE 12	
	Predictive + Shutdown	Remaining T3 (6 sub-tareas)	
8	11.1 Predictive Health	12.1 Compliance Watchdog	
	11.2 Shutdown Optimizer	12.2 Knowledge Curator	
	PARALELO 11.1 y 11.2	12.3 Spare Parts Forecaster	
		12.4 Autonomous RCA Initiator	
		12.5 Contractor + Benchmarking	
		12.6 Digital Twin + Energy	
		PARALELO 12.1-12.6	
9	11.3 Endpoints + Frontend	Endpoints + Frontend para 12.1-12.6	
	SERIE >>> 11.1+11.2	SERIE >>> cada sub-tarea	
	VERIFICAR T3 COMPLETO: 10 soluciones finales		

Resumen de Capacidad de Paralelizacion

Tier	Fases	Max canales paralelos	Tareas serie (cuello de botella)	Tareas paralelo
T0	0	3	1 (registrar main.py)	3
T1	1-6	6	8 (frontend espera backend)	14
T2	7-10	4	4 (frontend espera backend)	11
T3	11-12	8	2 (frontend espera backend)	8
Total		max 8 simultaneos	15 serie	36 paralelo

70% de las tareas son paralelizables. El cuello de botella sistematico es que el frontend de cada solucion debe esperar a que su backend este listo. Esto es inherente al flujo endpoint → servicio → UI.

Ruta Critica (camino mas largo)

0.1 Router → 0.4 Registrar → 6.1 Exec Report Svc → 6.3 PPTX Generator → 6.4 Frontend
(Fase mas larga del T1: 4 tareas en serie)

Duracion estimada ruta critica T1: ~4 dias

Duracion total T1 con paralelismo: ~5 dias (vs. ~20 dias en serie)

Vista de Gantt Simplificada



Leyenda Gantt: ##### = infra | ===== = backend | ===== = frontend/tests

Ultima actualizacion: 2026-04-06 Todas las actividades en estatus: PENDIENTE